In this chapter, you will learn how to implement an in-memory caching layer to improve the performance of a web application. You will deploy a complex web application called Discourse, a modern forum software application that uses Redis for caching. You will also learn how to scale a cache cluster and how to monitor and tweak performance.

Depending on your caching strategy, you can either populate the cache in real time or on demand. In the mobile game example, on demand means that if the leaderboard is not in the cache, the application asks the relational database and puts the result into the cache. Any subsequent request to the cache will result in a cache hit, meaning the data is found in the cache. This will be true until the duration of the TTL (time-to-live) value on the cached value expires. This strategy is called *lazy-loading* data from the primary data store. Additionally, we could have a job running in the background that queries the leaderboard from the relational database every minute and puts the result in the cache to populate the cache in advance.

The lazy-loading strategy (getting data on demand) is implemented like this:

1. The application writes data to the data store.
2. If the application wants to read the data, at a later time it makes a request to the caching layer.
3. If the caching layer does not contain the data, the application reads from the data store directly and puts the value into the cache, and also returns the value to the client.
4. Later, if the application wants to read the data again, it makes a request to the caching layer and finds the value.

This strategy comes with a problem. What if the data is changed while it is in the cache? The cache will still contain the old value. That’s why setting an appropriate TTL value is critical to ensure cache validity. Let’s say you apply a TTL of five minutes to your cached data: this means you accept that the data could be out of sync by up to five minutes with your primary database. Understanding the frequency of change for the underlying data and the effects out-of-sync data will have on the user experience is the first step of identifying the appropriate TTL value to apply. A common mistake some developers make is assuming that a few seconds of a cache TTL means that having a cache is not worthwhile. Remember that within those few seconds, millions of requests can be offloaded from your data store, speeding up your application and reducing database pressure. Performance testing your